



12. 循环神经网络实践



- Andrew Ng, Recurrent Neural Network

<https://www.bilibili.com/video/av88010803?p=1>

- 李宏毅，深度学习

<https://www.bilibili.com/video/BV1JE411g7XF>

建设单位： 大连理工大学

建设人： 孔雨秋



目录

CONTENT

01

注意力模型

02

Bleu得分

03

循环神经网络实践

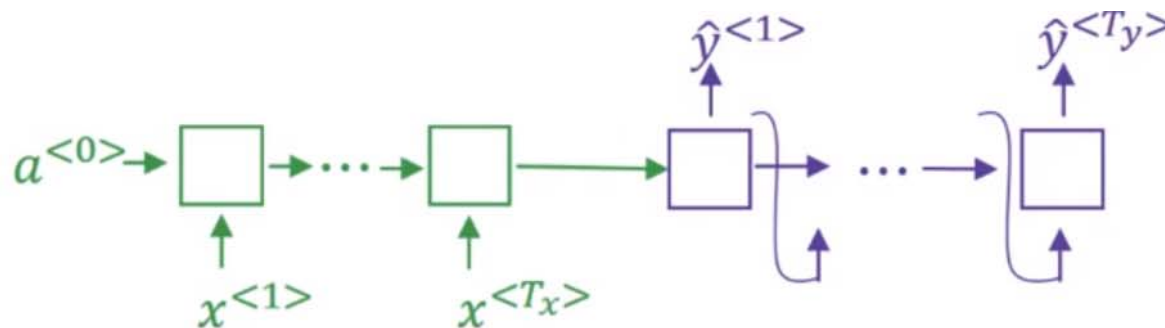
04

小结和资源



■ 机器翻译

- 文本序列越长，机器翻译的准确率越低。RNN无法记忆整个长句子。
- 注意力模型每次只翻译句子的一部分，将注意力分配给句子的不同部分。

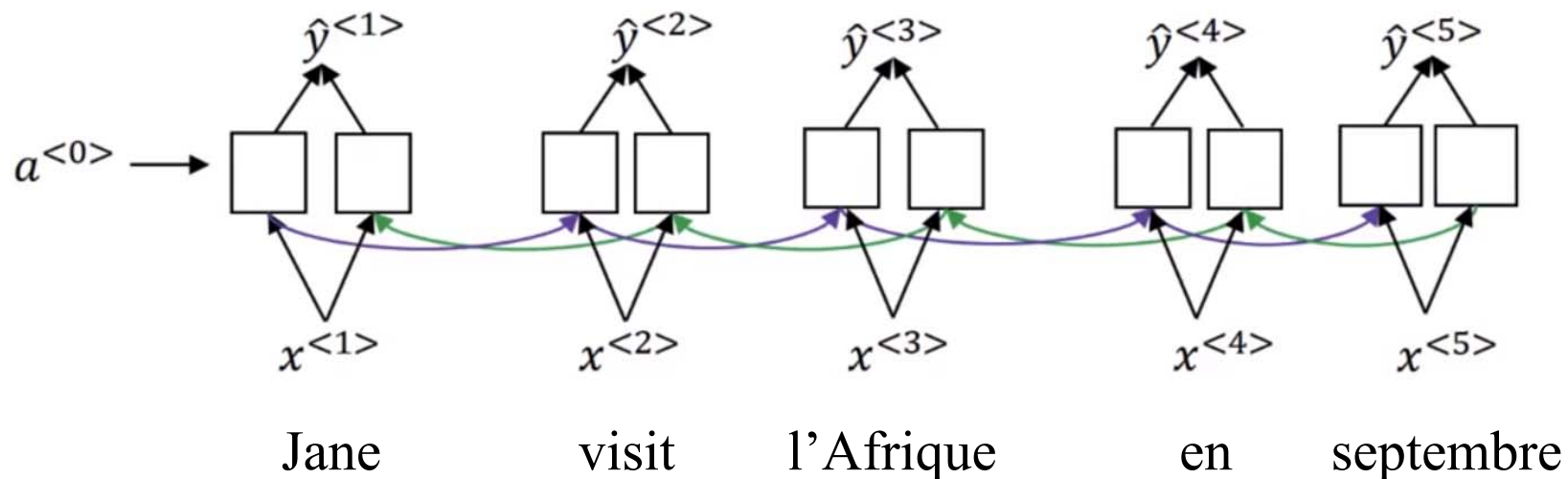


Jane s'est rendue en Afrique en septembre dernier, a apprécié la culture et a rencontré beaucoup de gens merveilleux; elle est revenue en parlant comment son voyage était merveilleux, et elle me tente d'y aller aussi.

Jane went to Africa last September, and enjoyed the culture and met many wonderful people; she came back raving about how wonderful her trip was, and is tempting me to go too.

■ 机器翻译

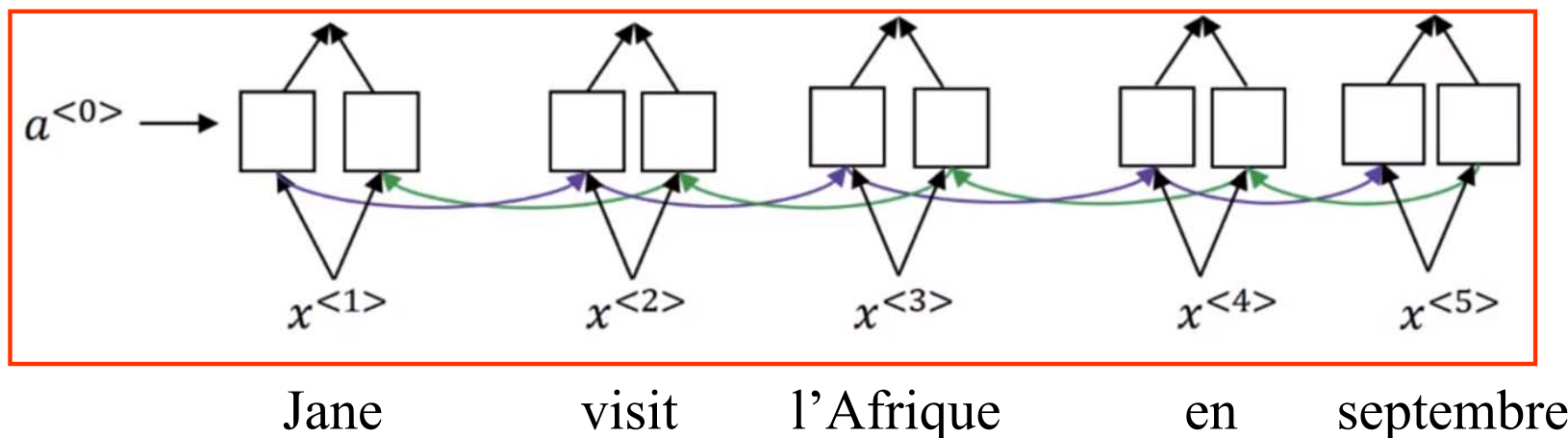
□ 注意力模型的直观解释



■ 机器翻译

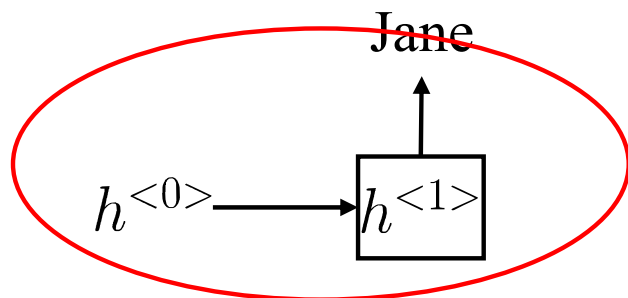
□ 注意力模型的直观解释

提取单词或周围单词的特征



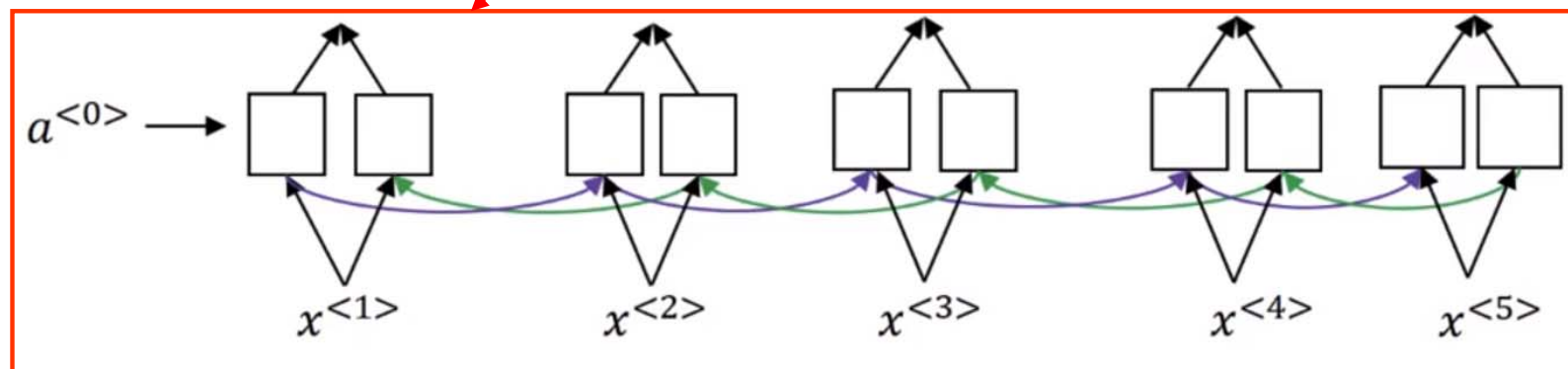
■ 机器翻译

□ 注意力模型的直观解释



□ 用另一个RNN输出翻译结果

□ 计算注意力权重，决定生成第一个单词时为每个输入单词分配多少注意力



Jane

visit

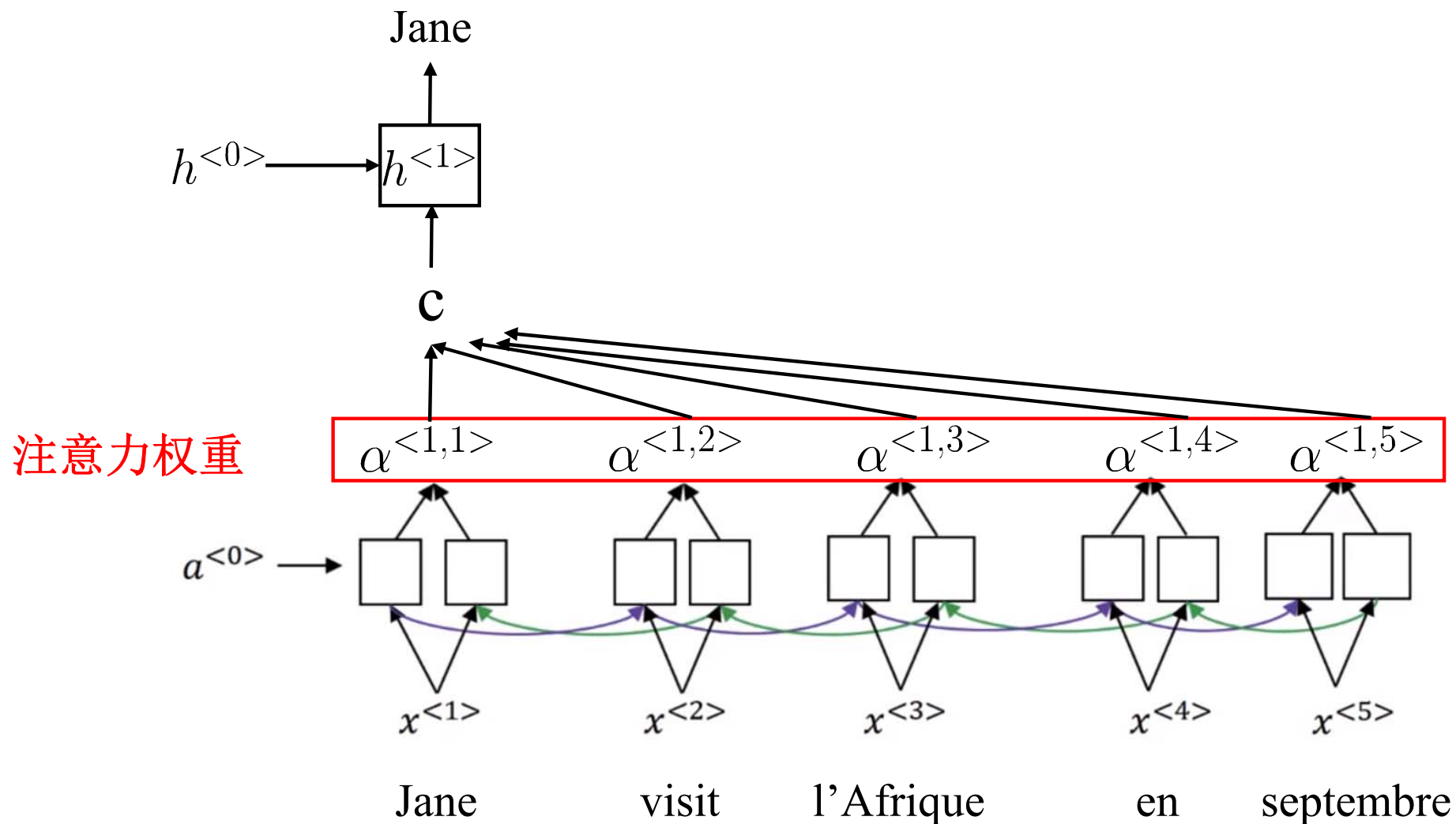
l'Afrique

en

septembre

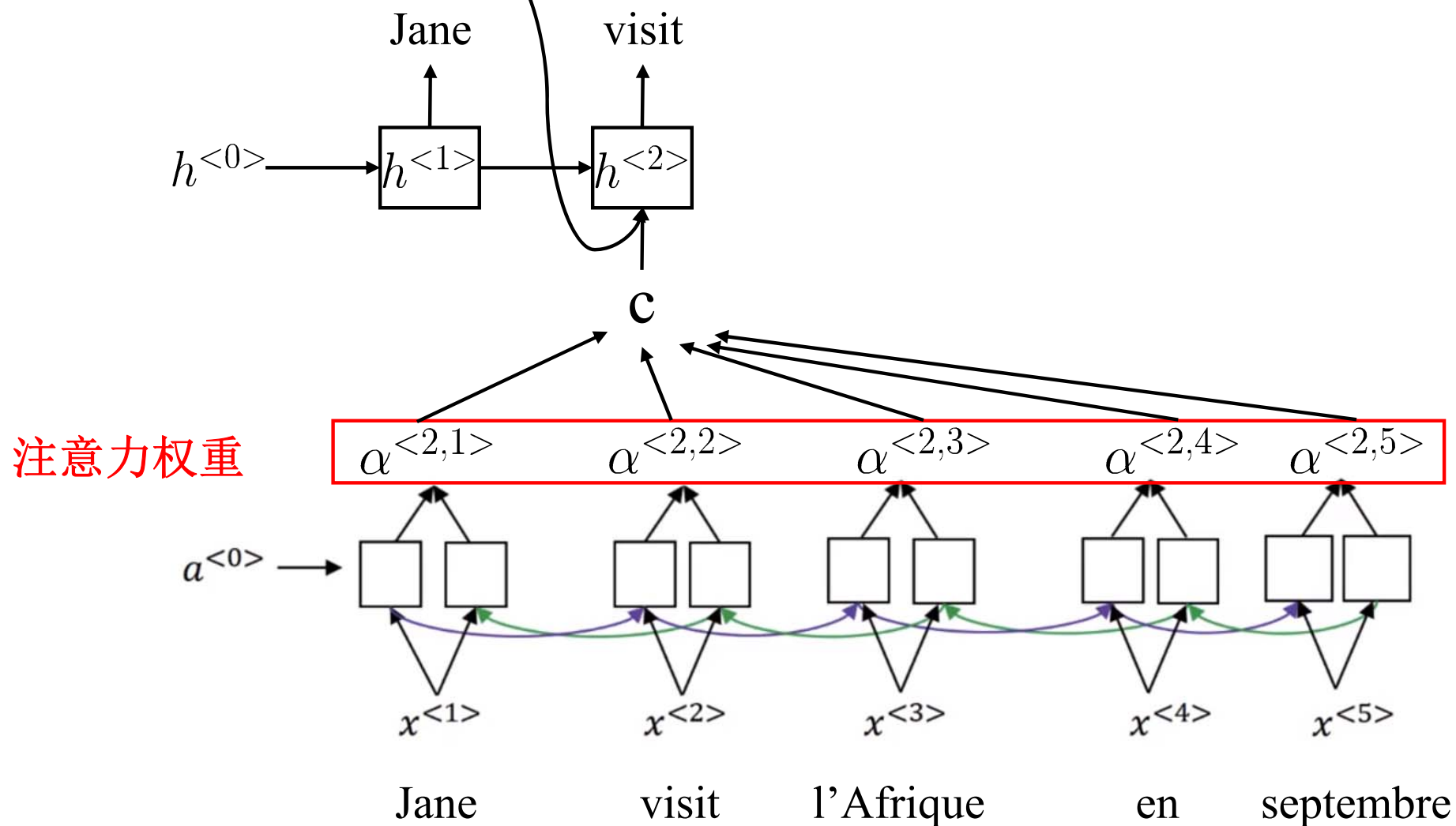
■ 机器翻译

□ 注意力模型的直观解释



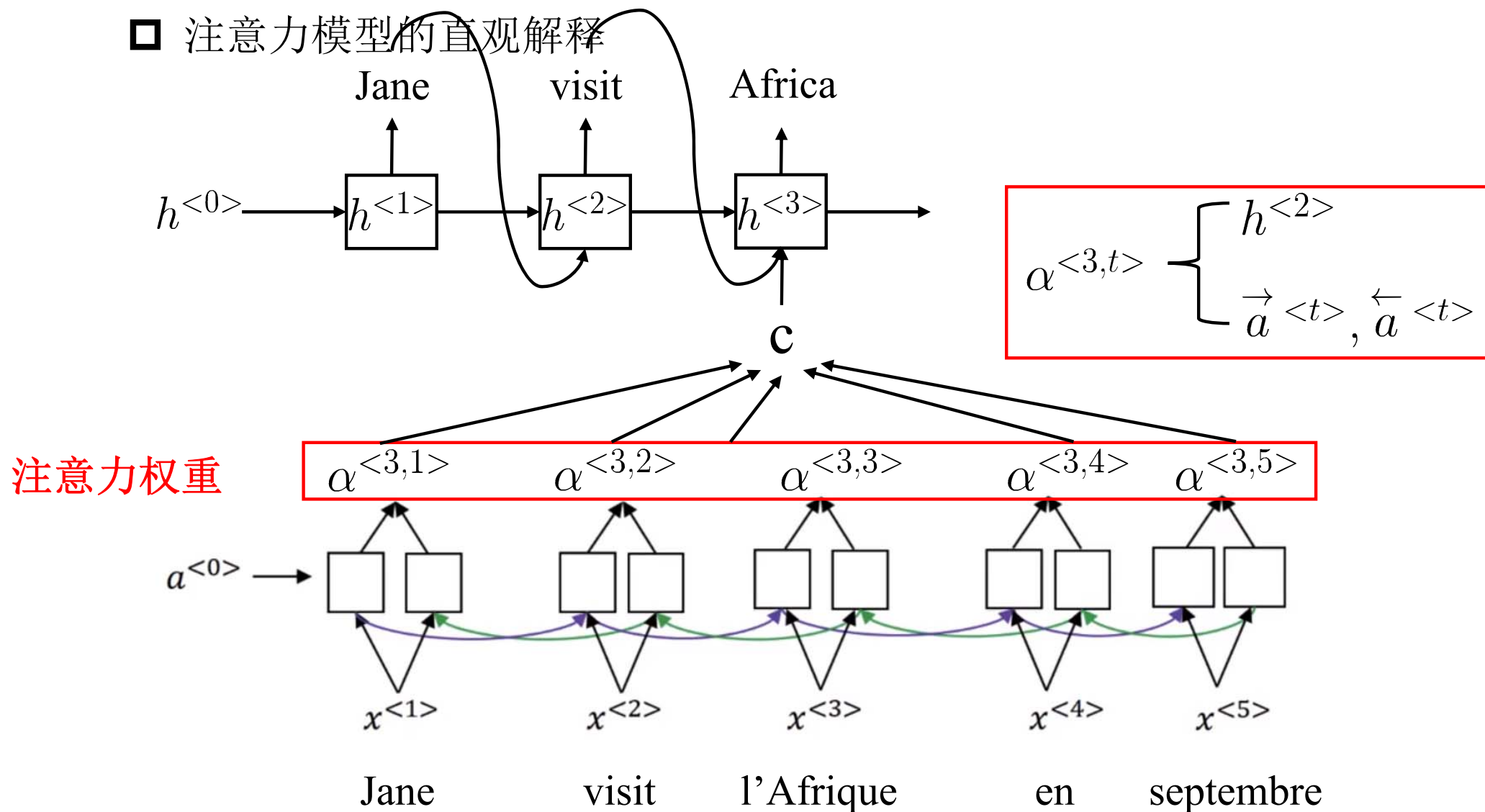
■ 机器翻译

□ 注意力模型的直观解释



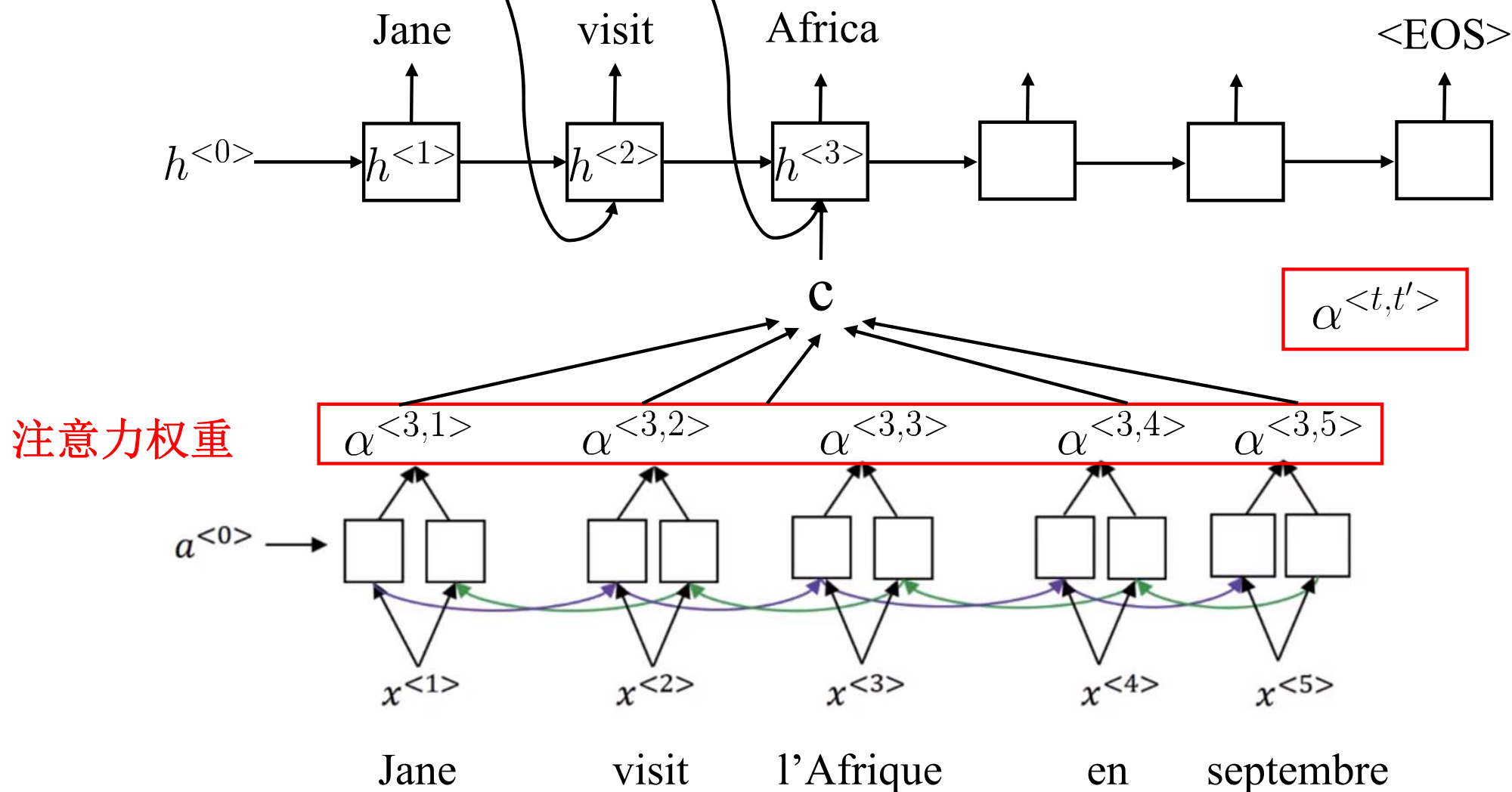
■ 机器翻译

□ 注意力模型的直观解释



■ 机器翻译

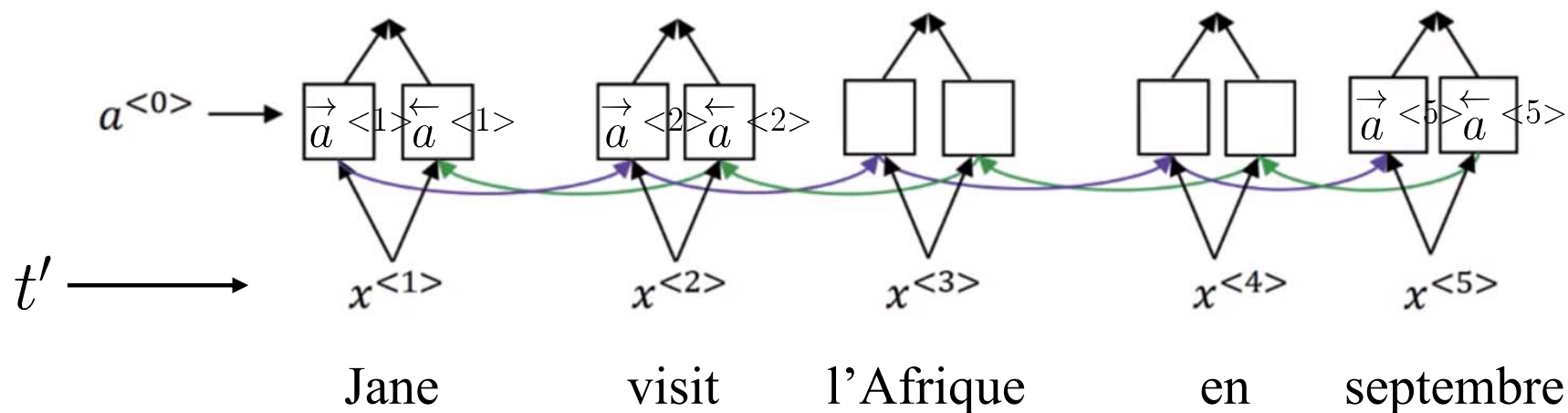
□ 注意力模型的直观解释



■ 机器翻译

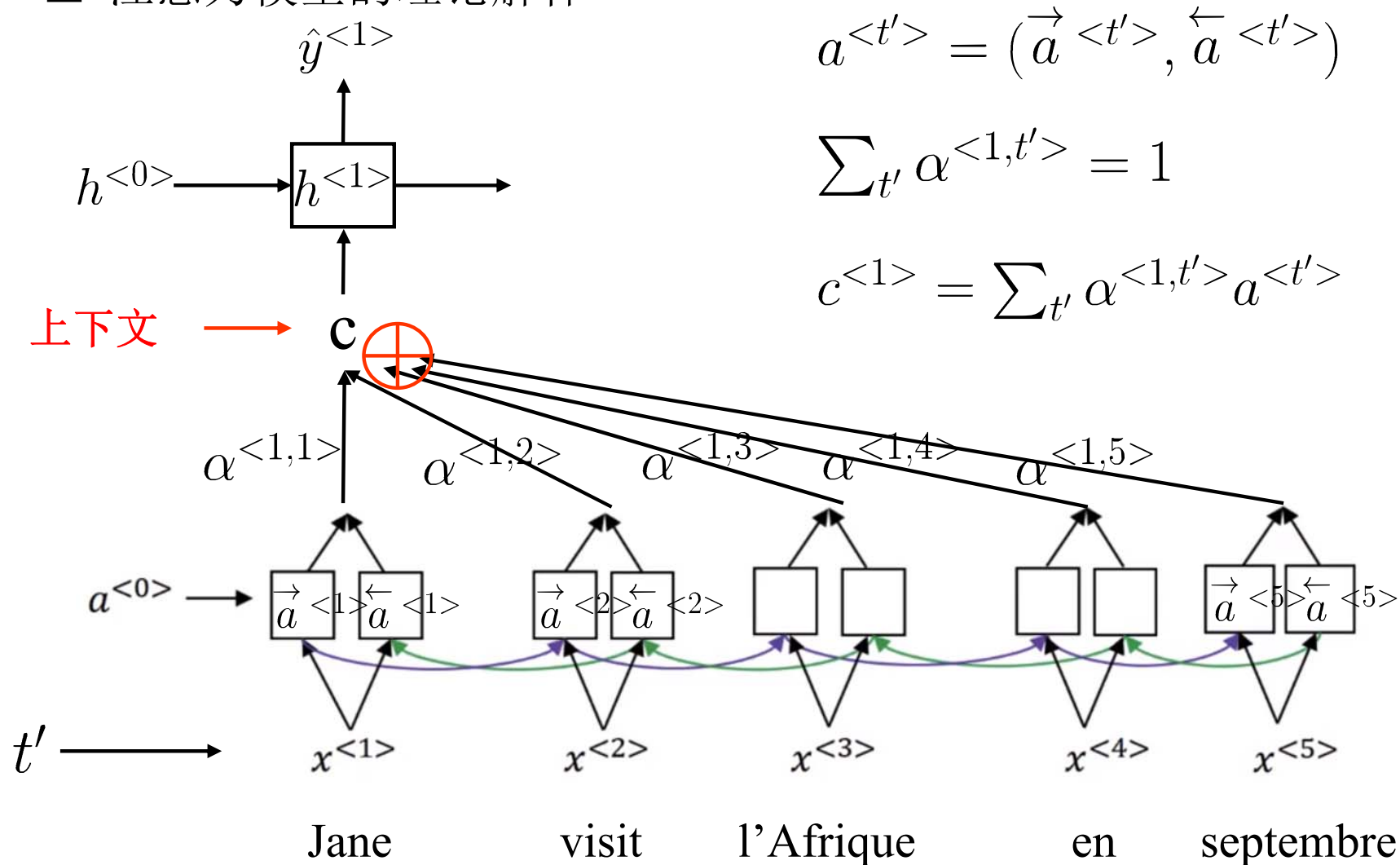
□ 注意力模型的理论解释

$$a^{<t'>} = (\vec{a}^{<t'>}, \overleftarrow{a}^{<t'>})$$



■ 机器翻译

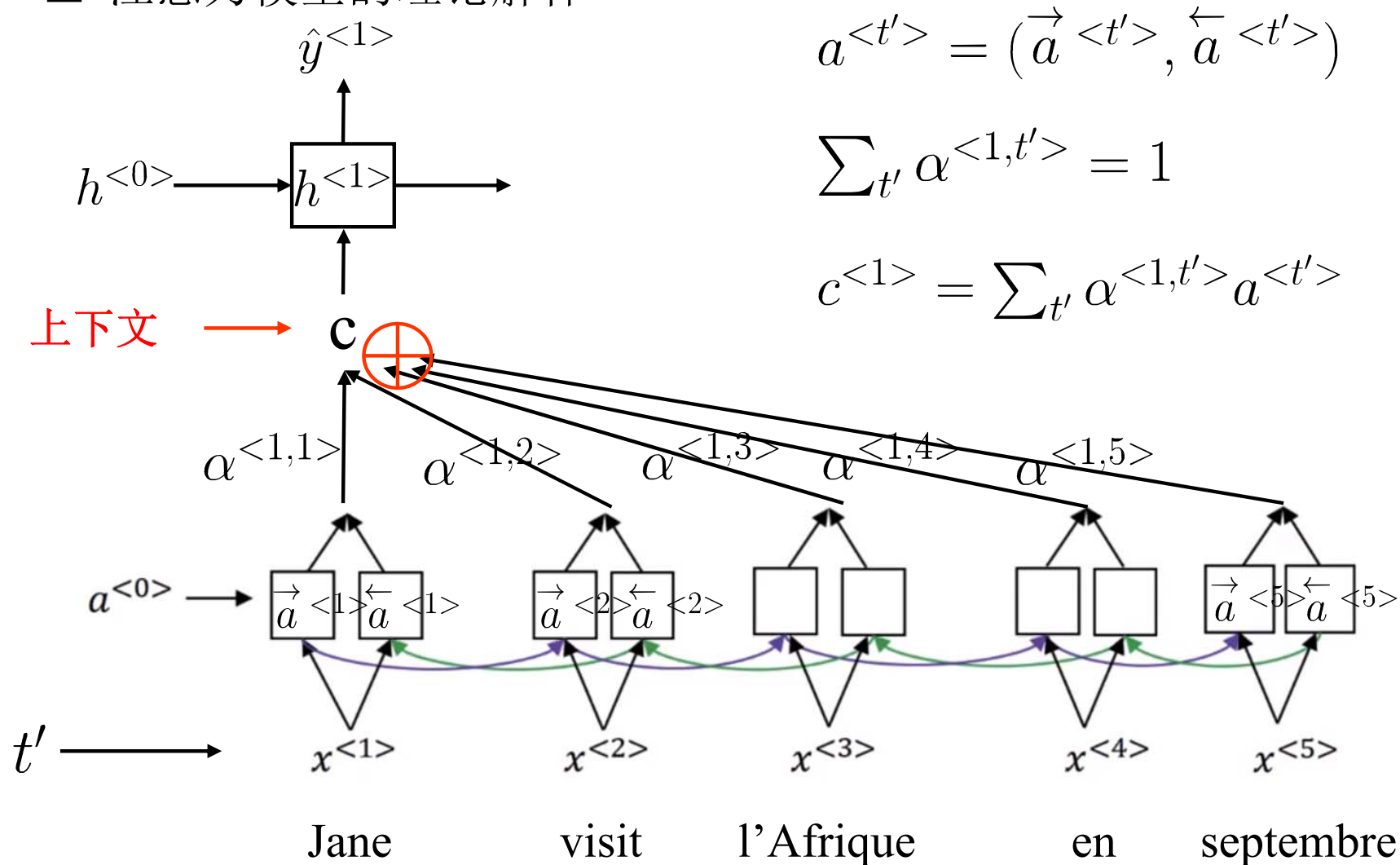
□ 注意力模型的理论解释



机器翻译

$\alpha^{<t,t'>}$ 预测 $\hat{y}^{<t>}$ 时分配给 $a^{<t'>}$ 的注意力

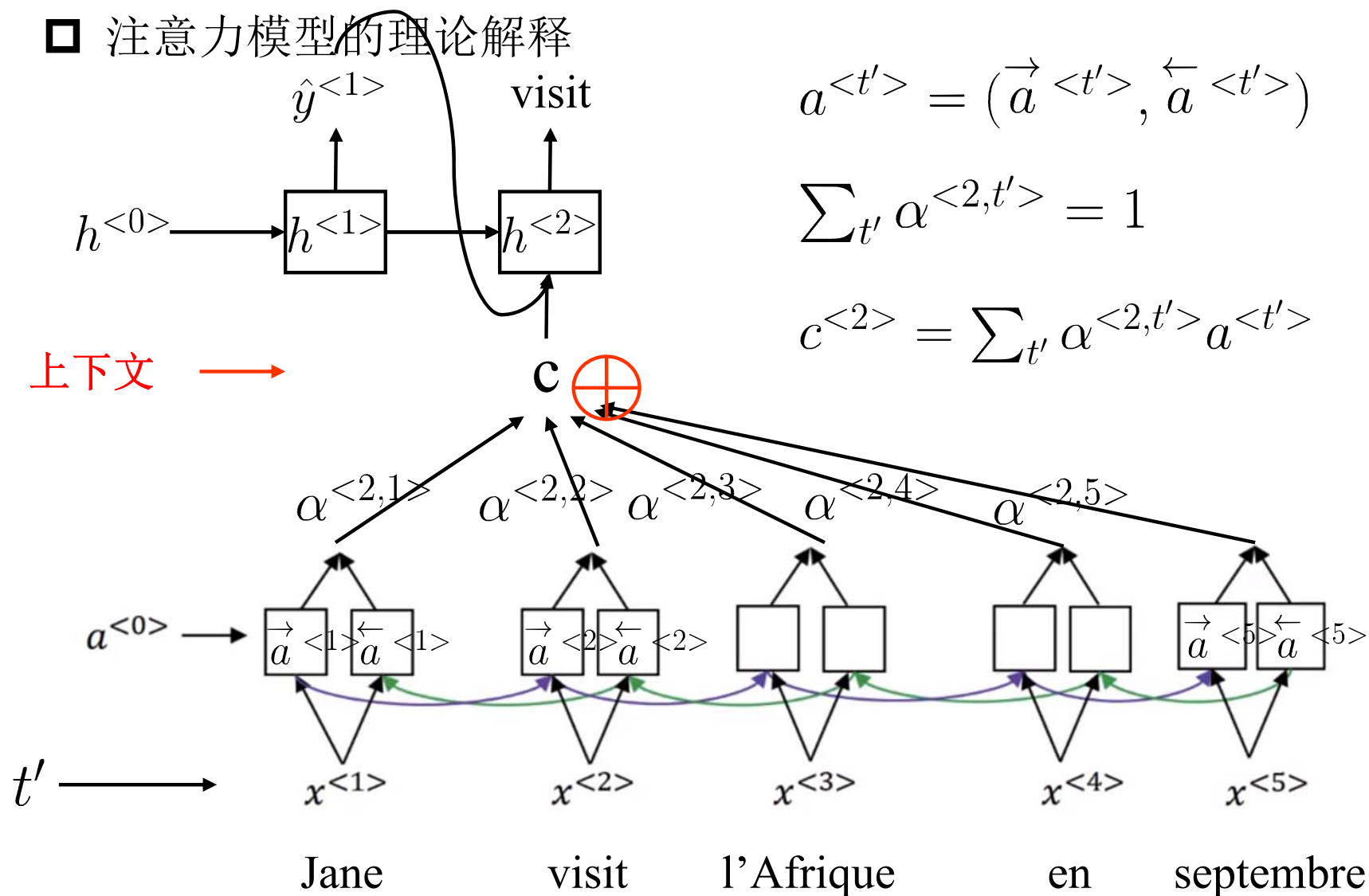
注意力模型的理论解释



■ 机器翻译

 $\alpha^{<t,t'>}$ 预测 $\hat{y}^{<t'>}$ 时分配给 $a^{<t'>}$ 的注意力

□ 注意力模型的理论解释



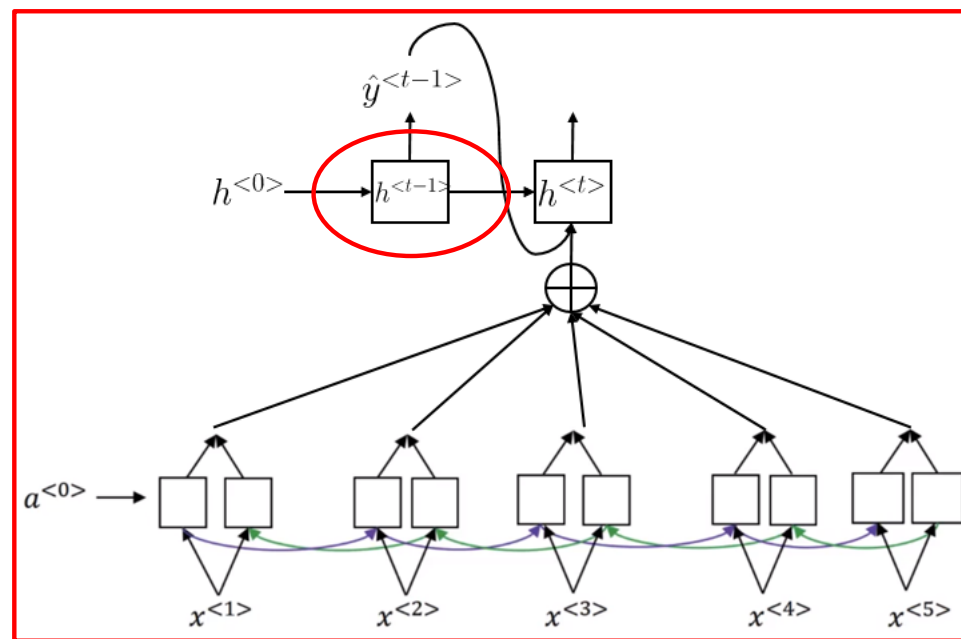
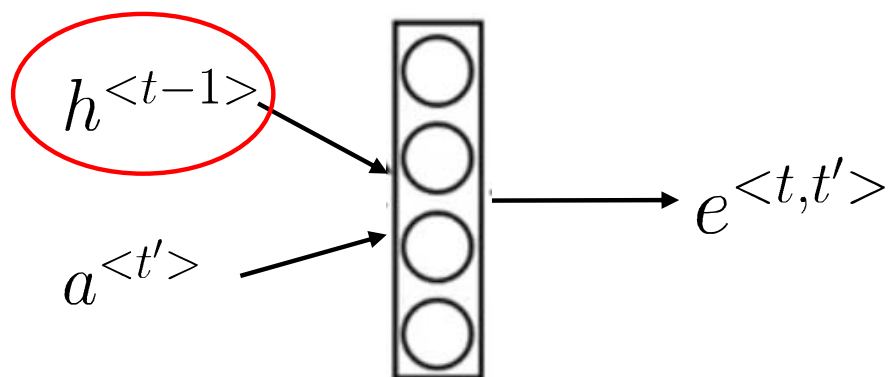
■ 机器翻译

□ 注意力模型的理论解释

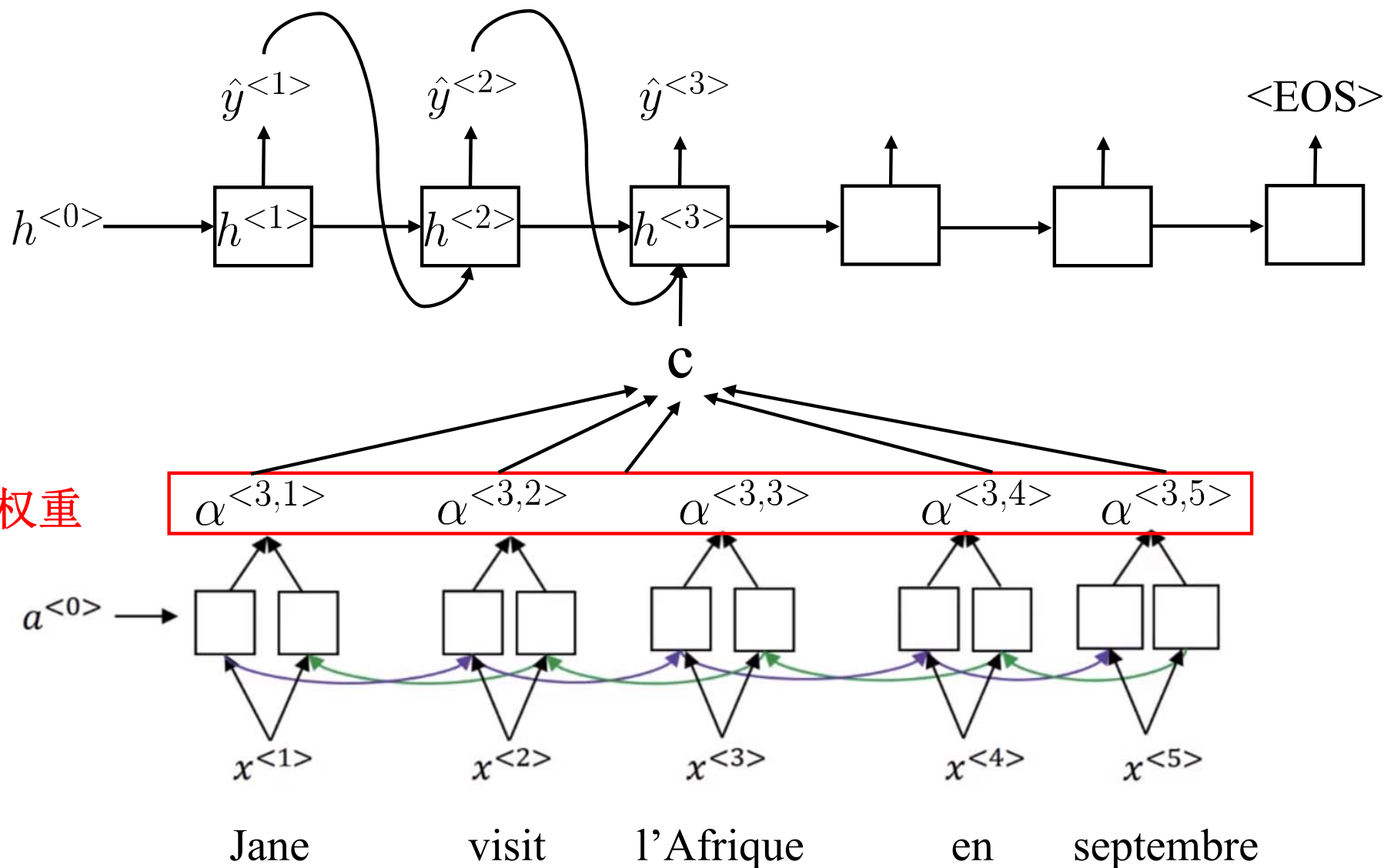
$\alpha^{<t,t'>}$ 预测 $\hat{y}^{<t>}$ 时分配给 $a^{<t'>}$ 的注意力

$$\alpha^{<t,t'>} = \frac{\exp(e^{<t,t'>})}{\sum_{t'=1}^{T_x} \exp(e^{<t,t'>})}$$

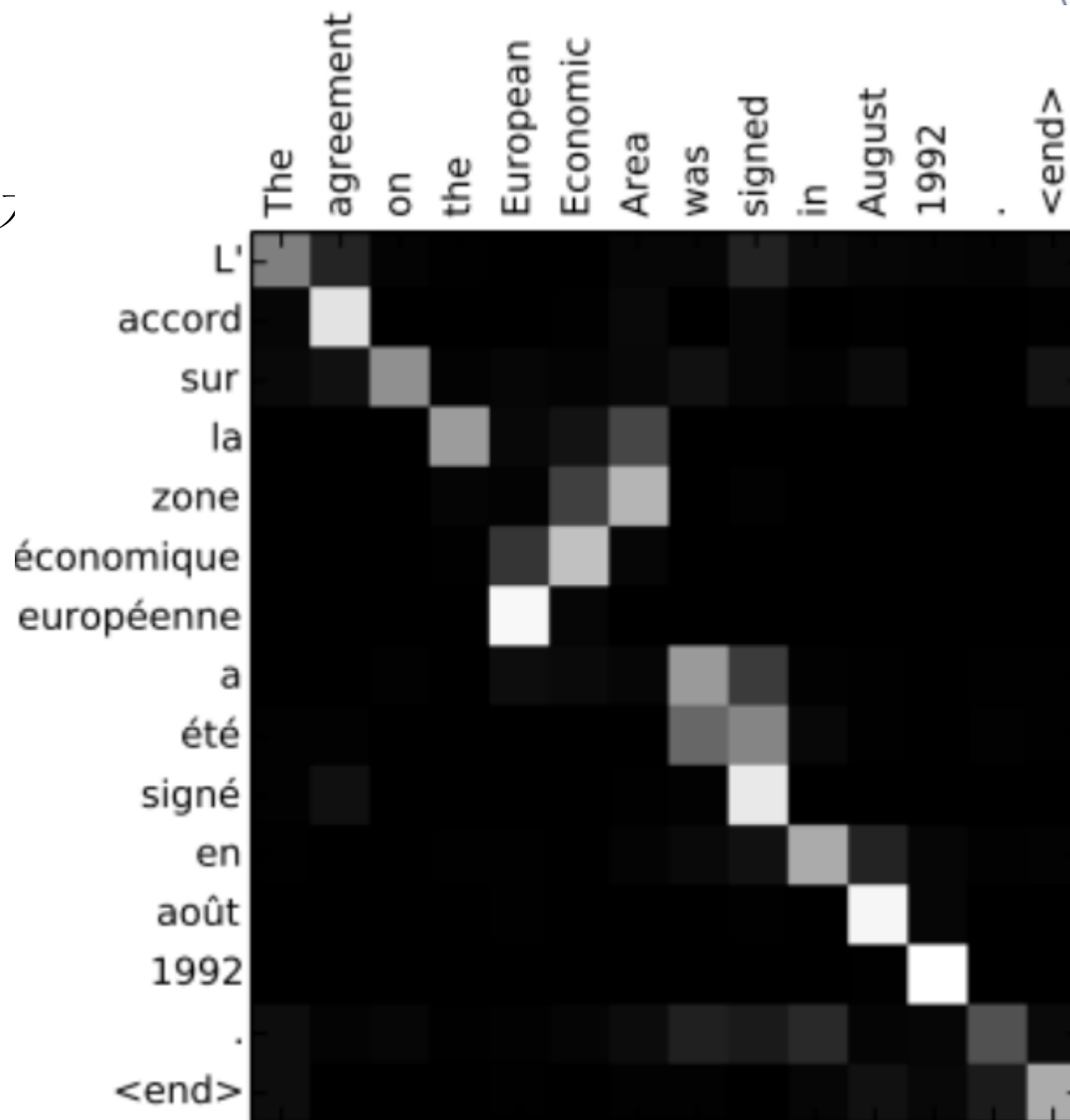
← **Softmax函数**，使注意力权重之和等于1



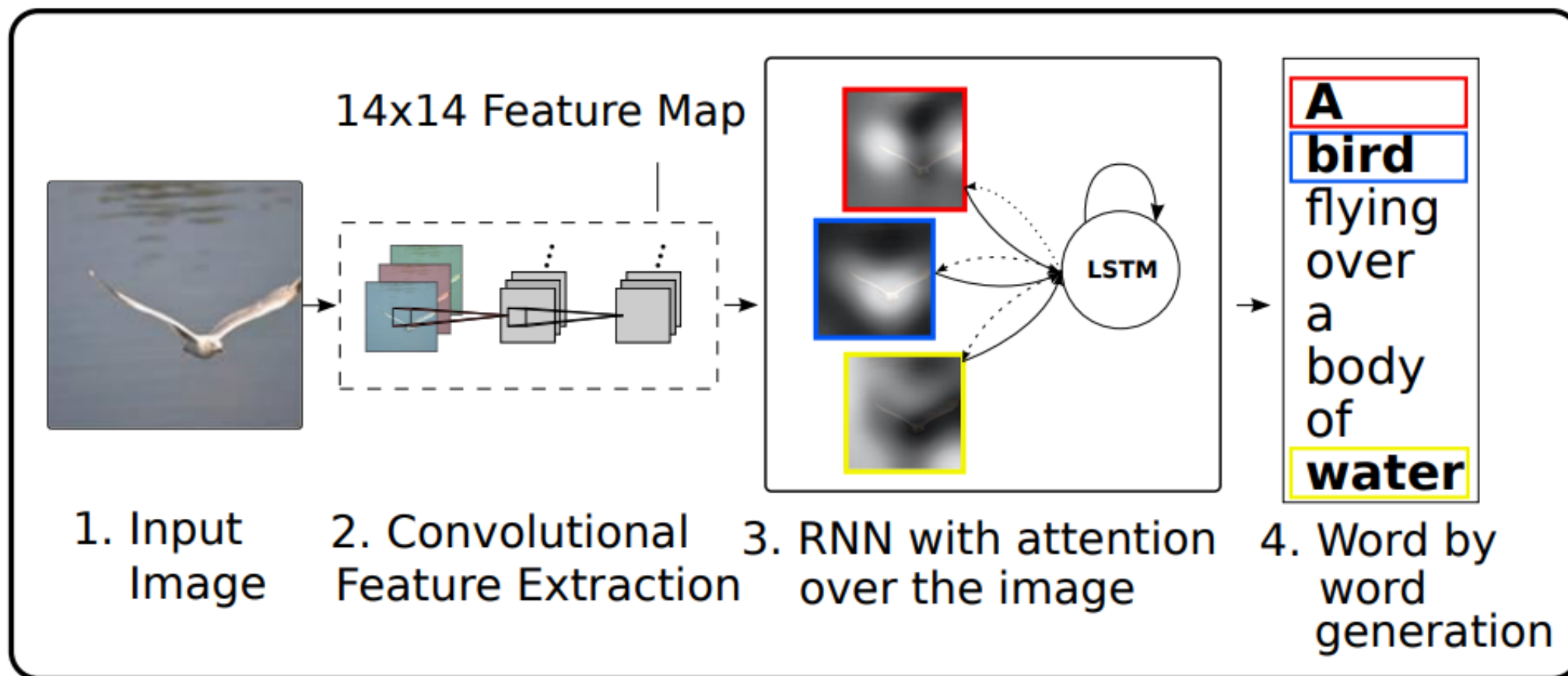
机器翻译



- 机器翻译
- 注意力



■ Image Caption



Show, Attend and Tell: Neural Image Caption Generation with Visual Attention. *Kelvin Xu et al.* Proceedings of the 32nd International Conference on Machine Learning, PMLR 37:2048-2057, 2015.



PART TWO

Bleu得分

Bleu score

Andrew Ng, Recurrent Neural Network

<https://www.bilibili.com/video/av88010803?p=1>





- Bleu得分：评估机器翻译系统

French: Le chat est sur le tapis.

Reference 1: The cat is on the mat.

Reference 2: There is a cat on the mat.

- Bleu得分：看机器翻译生成的词是否出现在人工翻译参考中





■ Bleu得分: 评估机器翻译系统

- 看机器翻译生成的词是否出现在人工翻译参考中

French: Le chat est sur le tapis.

Reference 1: The cat is on the mat.

Reference 2: There is a cat on the mat.

MT output: the the the the the the the.

Precision: 观察翻译的每个词是否出现在人工参考中



Precision=7/7





■ Bleu得分: 评估机器翻译系统

□ 看机器翻译生成的词是否出现在人工翻译参考中

French: Le chat est sur le tapis.

Reference 1: The cat is on the mat.

Reference 2: There is a cat on the mat.

MT output: the the the the the the the.

Precision: 观察翻译的每个词是否出现在人工参考中

Modified Precision: 每个单词的记分上限定为它在人工参考中出现的最高次数

$$\Rightarrow \text{Modified Precision} = 2/7 \quad \Rightarrow \text{Count}_{clip}(\text{"the"}) / \text{Count}(\text{"the"})$$





■ 二元词组的Bleu得分 (Bleu on bigrams)

Reference 1: The cat is on the mat.

Reference 2: There is a cat on the mat.

MT output: The cat the cat on the mat.

	$Count$	$Count_{clip}$
the cat	2	1
cat the	1	0
cat on	1	1
on the	1	1
the mat	1	1

➡ Modified Precision=4/6=2/3





■ n元词组的Bleu得分 (Bleu on n-grams)

Reference 1: The cat is on the mat.

Reference 2: There is a cat on the mat.

MT output: The cat the cat on the mat.

一元词组的精确度 $P_1 = \frac{\sum_{unigram \in \hat{y}} Count_{clip}(unigram)}{\sum_{unigram \in \hat{y}} Count(unigram)}$

n元词组的精确度 $P_n = \frac{\sum_{n-grams \in \hat{y}} Count_{clip}(n-grams)}{\sum_{n-grams \in \hat{y}} Count(n-grams)}$

$$Bleu = \exp\left(\frac{1}{4} \sum_{n=1}^4 P_n\right)$$





- BP (Brevity penalty, 简短惩罚) 惩罚因子：惩罚简短的输出

$$Bleu = \exp\left(\frac{1}{4} \sum_{n=1}^4 P_n\right)$$

$$\rightarrow Bleu = BP\left(\exp\left(\frac{1}{4} \sum_{n=1}^4 P_n\right)\right)$$

$$BP = \begin{cases} 1 & \text{if MT_output_length} > \text{reference_output_length} \\ \exp(1 - \text{reference_output_length} / \text{MT_output_length}), & \text{otherwise} \end{cases}$$



03

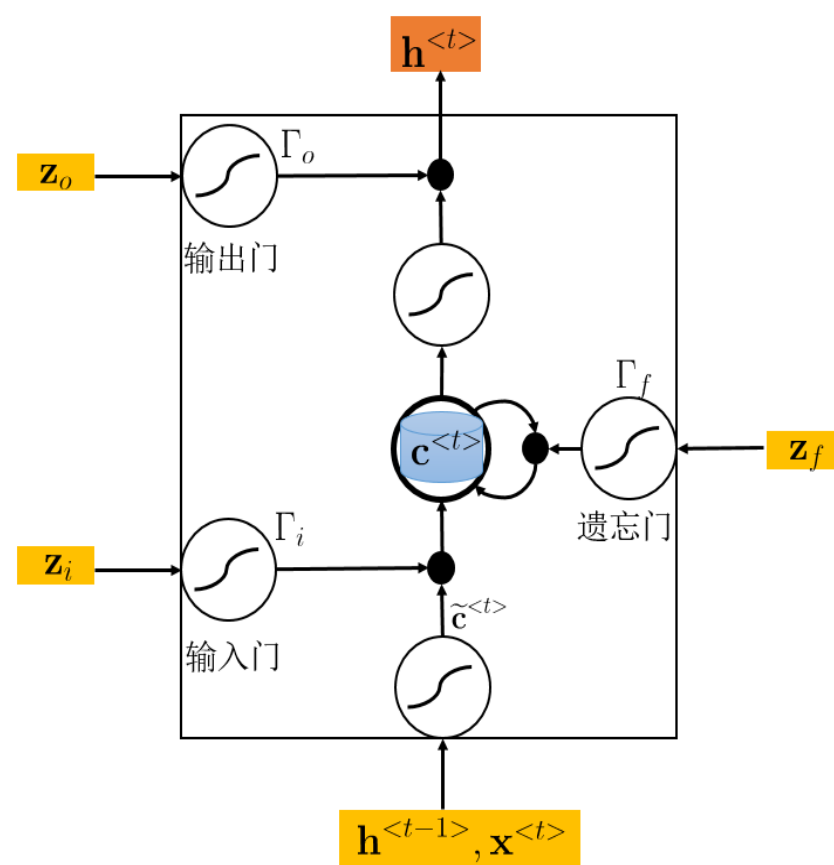
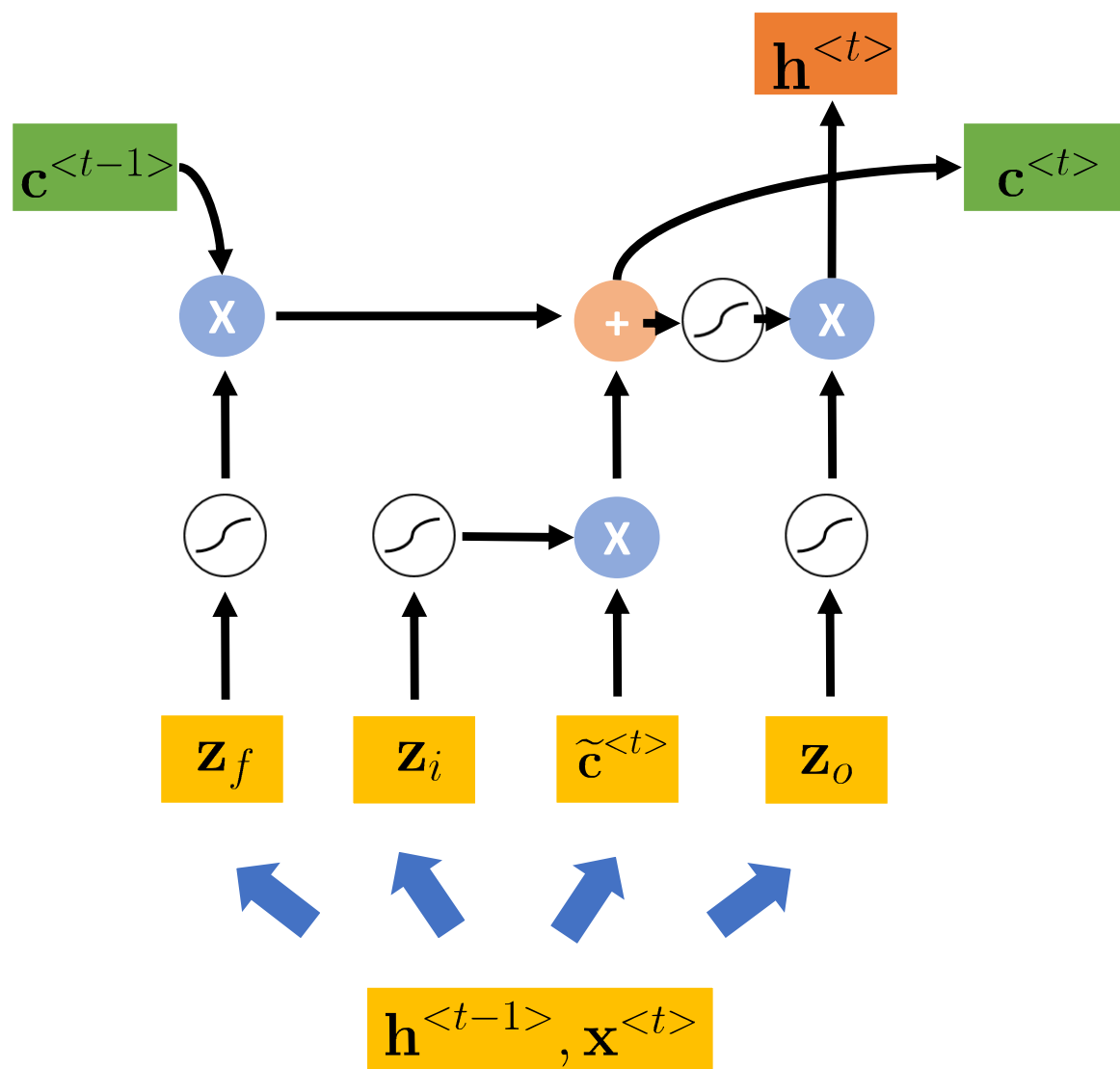
PART THREE

RNN生成周杰伦歌词

Lyrics generation

https://zh.d2l.ai/chapter_recurrent-neural-networks/lstm.html





$$\begin{aligned}\Gamma_f &= \sigma(\mathbf{W}_f[\mathbf{h}^{<t-1>}, \mathbf{x}^{<t>}] + \mathbf{b}_f) \\ \Gamma_i &= \sigma(\mathbf{W}_i[\mathbf{h}^{<t-1>}, \mathbf{x}^{<t>}] + \mathbf{b}_i) \\ \tilde{c}^{<t>} &= \tanh(\mathbf{W}_c[\mathbf{h}^{<t-1>}, \mathbf{x}^{<t>}] + \mathbf{b}_c) \\ c^{<t>} &= \Gamma_f * c^{<t-1>} + \Gamma_i * \tilde{c}^{<t>} \\ \Gamma_o &= \sigma(\mathbf{W}_o[\mathbf{h}^{<t-1>}, \mathbf{x}^{<t>}] + \mathbf{b}_o) \\ h^{<t>} &= \Gamma_o * \tanh(c^{<t>})\end{aligned}$$

```
num_inputs, num_hiddens, num_outputs = vocab_size, 256, vocab_size
print('will use', device)
```

```
def get_params():
    def _one(shape):
        ts = torch.tensor(np.random.normal(0, 0.01, size=shape), device=device, dtype=torch.float32)
        return torch.nn.Parameter(ts, requires_grad=True)
    def _three():
        return (_one((num_inputs, num_hiddens)),
                _one((num_hiddens, num_hiddens)),
                torch.nn.Parameter(torch.zeros(num_hiddens, device=device, dtype=torch.float32), requires_grad=True))

    W_xi, W_hi, b_i = _three() # 输入门参数
    W_xf, W_hf, b_f = _three() # 遗忘门参数
    W_xo, W_ho, b_o = _three() # 输出门参数
    W_xc, W_hc, b_c = _three() # 候选记忆细胞参数

    # 输出层参数
    W_hq = _one((num_hiddens, num_outputs))
    b_q = torch.nn.Parameter(torch.zeros(num_outputs, device=device, dtype=torch.float32), requires_grad=True)
    return nn.ParameterList([W_xi, W_hi, b_i, W_xf, W_hf, b_f, W_xo, W_ho, b_o, W_xc, W_hc, b_c, W_hq, b_q])
```

```
def lstm(inputs, state, params):
    [W_xi, W_hi, b_i, W_xf, W_hf, b_f, W_xo, W_ho, b_o, W_xc, W_hc, b_c,
     W_hq, b_q] = params
    (H, C) = state
    outputs = []
    for X in inputs:
        I = nd.sigmoid(nd.dot(X, W_xi) + nd.dot(H, W_hi) + b_i)
        F = nd.sigmoid(nd.dot(X, W_xf) + nd.dot(H, W_hf) + b_f)
        O = nd.sigmoid(nd.dot(X, W_xo) + nd.dot(H, W_ho) + b_o)
        C_tilda = nd.tanh(nd.dot(X, W_xc) + nd.dot(H, W_hc) + b_c)
        C = F * C + I * C_tilda
        H = O * C.tanh()
        Y = nd.dot(H, W_hq) + b_q
        outputs.append(Y)
    return outputs, (H, C)
```

1

GRU

LSTM

$$\Gamma_f = \sigma(\mathbf{W}_f[\mathbf{h}^{<t-1>}, \mathbf{x}^{<t>}] + \mathbf{b}_f)$$

$$\Gamma_i = \sigma(\mathbf{W}_i[\mathbf{h}^{<t-1>}, \mathbf{x}^{<t>}] + \mathbf{b}_i)$$

$$\tilde{\mathbf{c}}^{<t>} = \tanh(\mathbf{W}_c[\mathbf{h}^{<t-1>}, \mathbf{x}^{<t>}] + \mathbf{b}_c)$$

$$\mathbf{c}^{<t>} = \Gamma_f * \mathbf{c}^{<t-1>} + \Gamma_i * \tilde{\mathbf{c}}^{<t>}$$

$$\Gamma_o = \sigma(\mathbf{W}_o[\mathbf{h}^{<t-1>}, \mathbf{x}^{<t>}] + \mathbf{b}_o)$$

$$\mathbf{h}^{<t>} = \Gamma_o * \tanh(\mathbf{c}^{<t>})$$

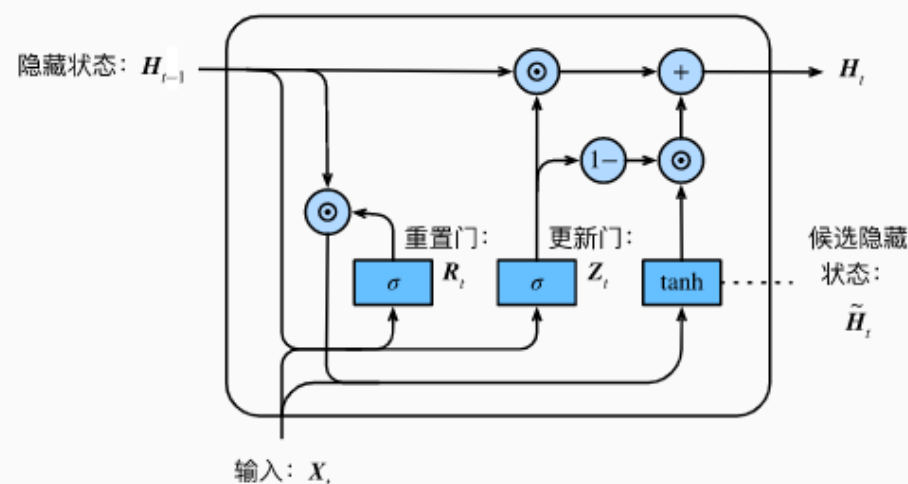
GRU

$$\Gamma_r = \sigma(\mathbf{W}_r[\mathbf{h}^{<t-1>}, \mathbf{x}^{<t>}] + \mathbf{b}_r)$$

$$\Gamma_u = \sigma(\mathbf{W}_r[\mathbf{h}^{<t-1>}, \mathbf{x}^{<t>}] + \mathbf{b}_u)$$

$$\tilde{\mathbf{c}}^{<t>} = \tanh(\mathbf{W}_h[\Gamma_r \mathbf{c}^{<t-1>}, \mathbf{x}^{<t>}] + \mathbf{b}_h)$$

$$\mathbf{h}^{<t>} = \Gamma_u * \mathbf{h}^{<t-1>} + (1 - \Gamma_u) * \tilde{\mathbf{c}}^{<t>}$$



```
num_inputs, num_hiddens, num_outputs = vocab_size, 256, vocab_size
print('will use', device)
```

```
def get_params():
    def _one(shape):
        ts = torch.tensor(np.random.normal(0, 0.01, size=shape), device=device, dtype=torch.float32)
        return torch.nn.Parameter(ts, requires_grad=True)
    def _three():
        return (_one((num_inputs, num_hiddens)),
                _one((num_hiddens, num_hiddens)),
                torch.nn.Parameter(torch.zeros(num_hiddens, device=device, dtype=torch.float32), requires_grad=True))

    W_xz, W_hz, b_z = _three()  # 更新门参数
    W_xr, W_hr, b_r = _three()  # 重置门参数
    W_xh, W_hh, b_h = _three()  # 候选隐藏状态参数

    # 输出层参数
    W_hq = _one((num_hiddens, num_outputs))
    b_q = torch.nn.Parameter(torch.zeros(num_outputs, device=device, dtype=torch.float32), requires_grad=True)
    return nn.ParameterList([W_xz, W_hz, b_z, W_xr, W_hr, b_r, W_xh, W_hh, b_h, W_hq, b_q])
```

```
def gru(inputs, state, params):
    W_xz, W_hz, b_z, W_xr, W_hr, b_r, W_xh, W_hh, b_h, W_hq, b_q = params
    H, = state
    outputs = []
    for X in inputs:
        Z = torch.sigmoid(torch.matmul(X, W_xz) + torch.matmul(H, W_hz) + b_z)
        R = torch.sigmoid(torch.matmul(X, W_xr) + torch.matmul(H, W_hr) + b_r)
        H_tilda = torch.tanh(torch.matmul(X, W_xh) + torch.matmul(R * H, W_hh) + b_h)
        H = Z * H + (1 - Z) * H_tilda
        Y = torch.matmul(H, W_hq) + b_q
        outputs.append(Y)
    return outputs, (H,)
```



■ 语言模型数据集（歌词）

□ 收集了周杰伦从第一张专辑《Jay》到第十张专辑《跨时代》中的歌词

想要有直升机
 想要和你飞到宇宙去
 想要和你融化在一起
 融化在宇宙里
 我每天每天每天在想想想想
 这样的甜蜜
 让我开始相信命运
 感谢地心引力
 让我碰到你
 漂亮的让我面红的可爱女人
 温柔的让我心疼的可爱女人
 透明的让我感动的可爱女人
 坏坏的让我疯狂的可爱女人
 坏坏的让我疯狂的可爱女人
 漂亮的让我面红的可爱女人
 温柔的让我心疼的可爱女人
 透明的让我感动的可爱女人
 坏坏的让我疯狂的可爱女人
 坏坏的让我疯狂的可爱女人
 想要有直升机
 想要和你飞到宇宙去
 想要和你融化在一起

你的完美主义 太彻底
 让我连恨都难以下笔
 将真心抽离写成日记 像是一场默剧
 你的完美主义 太彻底
 分手的话像语言暴力
 我已无能为力再提起 决定中断熟悉
 然后在这里 不限日期
 然后将过去 慢慢温习
 让我爱上你 那场悲剧
 是你完美演出的一场戏
 宁愿心碎哭泣 再狠狠忘记 你爱过我的证据
 让晶莹的泪滴 闪烁成回忆 伤人的美丽
 你的完美主义 太彻底
 让我连恨都难以下笔
 将真心抽离写成日记 像是一场默剧
 你的完美主义 太彻底
 分手的话像语言暴力
 我已无能为力再提起 决定中断熟悉
 周杰伦 周杰伦
 一步两步三步四步望著天 看星星
 一颗两颗三颗四颗 连成线一步两步三步四步望著天 看星星
 一颗两颗三颗四颗 连成线 乖著风 游荡在蓝天边





```
def load_data_jay_lyrics():  
    """加载周杰伦歌词数据集"""  
    with zipfile.ZipFile('../data/jaychou_lyrics.txt.zip') as zin:  
        with zin.open('jaychou_lyrics.txt') as f:  
            corpus_chars = f.read().decode('utf-8')  
    corpus_chars = corpus_chars.replace('\n', ' ').replace('\r', ' ')  
    corpus_chars = corpus_chars[0:10000]  
    idx_to_char = list(set(corpus_chars))  
    char_to_idx = dict([(char, i) for i, char in enumerate(idx_to_char)])  
    vocab_size = len(char_to_idx)  
    corpus_indices = [char_to_idx[char] for char in corpus_chars]  
    return corpus_indices, char_to_idx, idx_to_char, vocab_size  
  
.....  
(corpus_indices, char_to_idx, idx_to_char, vocab_size) = load_data_jay_lyrics()
```





```
(corpus_indices, char_to_idx, idx_to_char, vocab_size) = load_data_jay_lyrics()
```

```
(Pdb) corpus_indices
```

```
[857, 629, 154, 583, 473, 204, 913, 857, 629, 347, 948, 366, 558, 75  
7, 158, 15, 913, 857, 629, 347, 948, 922, 367, 865, 379, 985, 913, 9  
22, 367, 865, 757, 158, 622, 913, 213, 477, 290, 477, 290, 477, 290,  
865, 857, 857, 857, 857, 177, 948, 913, 174, 808, 300, 559, 284, 91  
3, 55, 213, 939, 396, 53, 120, 311, 971, 493, 913, 390, 112, 126, 89  
4, 205, 708, 913, 55, 213, 465, 558, 948, 913, 1003, 325, 300, 55, 2
```

```
(Pdb) char_to_idx
```

```
{'鼻': 0, '坊': 1, '和': 2, '风': 3, '狠': 4, '旋': 5, '该': 6, '默':  
: 7, '争': 8, '砖': 9, '准': 10, '牌': 11, '驳': 12, '练': 13, 'u':  
14, '脉': 15, '黄': 16, '枚': 17, '偷': 18, '誓': 19, '萨': 20, '物':  
: 21, '荡': 22, '忠': 23, '瓦': 24, '王': 25, '头': 26, '甜': 27, '  
活': 28, '留': 29, '升': 30, '横': 31, '热': 32, '鸠': 33, '太': 34,  
'什': 35, '木': 36, '沼': 37, '骷': 38, '生': 39, '呼': 40, '转': 4
```

```
(Pdb) idx_to_char
```

```
['鼻', '坊', '和', '风', '狠', '旋', '该', '默', '争', '砖', '准', '  
牌', '驳', '练', 'u', '脉', '黄', '枚', '偷', '誓', '萨', '物', '荡',  
, '忠', '瓦', '王', '头', '甜', '活', '留', '升', '横', '热', '鸠',  
'太', '什', '木', '沼', '骷', '生', '呼', '转', '映', '更', '爵', '  
味', '废', '告', '林', '传', '秒', '乘', '板', '弃', '绪', '金', '啃',  
, '场', '踢', '而', '彻', '迹', '必', '染', '胖', '惚', '任', '刻',
```

```
(Pdb) vocab_size
```

```
1027
```

```
num_epochs, num_steps, batch_size, lr, clipping_theta = 160, 35, 32, 1e-2, 1e2
pred_period, pred_len, prefixes = 40, 50, ['分开', '不分开']
num_inputs, num_hiddens, num_outputs = vocab_size, 256, vocab_size

rnn_unit = "LSTM"
if rnn_unit == "LSTM":
    rnn_layer = nn.LSTM(input_size=vocab_size, hidden_size=num_hiddens)
elif rnn_unit == "GRU":
    rnn_layer = nn.GRU(input_size=vocab_size, hidden_size=num_hiddens)
```



```
class RNNModel(nn.Module):
    def __init__(self, rnn_layer, vocab_size):
        super(RNNModel, self).__init__()
        self.rnn = rnn_layer
        self.hidden_size = rnn_layer.hidden_size * (2 if rnn_layer.bidirectional else 1)
        self.vocab_size = vocab_size
        self.dense = nn.Linear(self.hidden_size, vocab_size)
        self.state = None

    def forward(self, inputs, state): # inputs: (batch, seq_len)
        # 获取one-hot向量表示
        X = to_onehot(inputs, self.vocab_size) # X是个List
        Y, self.state = self.rnn(torch.stack(X), state)
        # 全连接层会首先将Y的形状变成(num_steps * batch_size, num_hiddens) · 它的输出
        # 形状为(num_steps * batch_size, vocab_size)
        output = self.dense(Y.view(-1, Y.shape[-1]))
        return output, self.state
```





```
def train_and_predict_rnn_pytorch(model, vocab_size, device,
                                   corpus_indices, idx_to_char, char_to_idx,
                                   num_epochs, num_steps, lr, clipping_theta,
                                   batch_size, pred_period, pred_len, prefixes):
    loss = nn.CrossEntropyLoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)
    model.to(device)
    state = None
    for epoch in range(num_epochs):
        l_sum, n, start = 0.0, 0, time.time()
        data_iter = data_iter_consecutive(corpus_indices, batch_size, num_steps, device) # 相邻采样
        for X, Y in data_iter:
            if state is not None:
                # 使用detach函数从计算图分离隐藏状态, 这是为了
                # 使模型参数的梯度计算只依赖一次迭代读取的小批量序列(防止梯度计算开销太大)
                if isinstance(state, tuple): # LSTM, state:(h, c)
                    state = (state[0].detach(), state[1].detach())
                else:
                    state = state.detach()

            (output, state) = model(X, state) # output: 形状为(num_steps * batch_size, vocab_size)

            # Y的形状是(batch_size, num_steps), 转置后再变成长度为
            # batch * num_steps 的向量, 这样跟输出的行一一对应
            y = torch.transpose(Y, 0, 1).contiguous().view(-1)
            l = loss(output, y.long())

        optimizer.zero_grad()
        l.backward()
```




```
def train_and_predict_rnn_pytorch(model, vocab_size, device,
                                   corpus_indices, idx_to_char, char_to_idx,
                                   num_epochs, num_steps, lr, clipping_theta,
                                   batch_size, pred_period, pred_len, prefixes):
    loss = nn.CrossEntropyLoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)
    model.to(device)
    state = None
    for epoch in range(num_epochs):
        l_sum, n, start = 0.0, 0, time.time()
        data_iter = data_iter_consecutive(corpus_indices, batch_size, num_steps, device) # 相邻采样
        for X, Y in data_iter:
            if state is not None:
                # 使用detach函数从计算图分离隐藏状态, 这是为了
                # 使模型参数的梯度计算只依赖一次迭代读取的小批量序列(防止梯度计算开销太大)
                if isinstance(state, tuple): # LSTM, state:(h, c)
                    state = (state[0].detach(), state[1].detach())
                else:
                    state = state.detach()

            (output, state) = model(X, state) # output: 形状为(num_steps * batch_size, vocab_size)

            # Y的形状是(batch_size, num_steps), 转置后再变成长度为
            # batch * num_steps 的向量, 这样跟输出的行一一对应
            y = torch.transpose(Y, 0, 1).contiguous().view(-1)
            l = loss(output, y.long())

        optimizer.zero_grad()
        l.backward()
```



```
optimizer.zero_grad()
l.backward()
# 梯度裁剪
grad_clipping(model.parameters(), clipping_theta, device)
optimizer.step()
l_sum += l.item() * y.shape[0]
n += y.shape[0]

try:
    perplexity = math.exp(l_sum / n)
except OverflowError:
    perplexity = float('inf')
if (epoch + 1) % pred_period == 0:
    print('epoch %d, perplexity %f, time %.2f sec' % (
        epoch + 1, perplexity, time.time() - start))
    for prefix in prefixes:
        print('-', predict_rnn_pytorch(
            prefix, pred_len, model, vocab_size, device, idx_to_char,
            char_to_idx))
```



- 困惑度(perplexity): 评价语言模型的好坏。困惑度是对交叉熵损失函数做指数运算后得到的值²
- 最佳情况下，模型总是把标签类别的概率预测为1，此时困惑度为1；
- 最坏情况下，模型总是把标签类别的概率预测为0，此时困惑度为正无穷；
- 基线情况下，模型总是预测所有类别的概率都相同，此时困惑度为类别个数。

显然，任何一个有效模型的困惑度必须小于类别个数。在本例中，困惑度必须小于词典大小`vocab_size`。





```
def predict_rnn_pytorch(prefix, num_chars, model, vocab_size, device, idx_to_char,
                        char_to_idx):
    state = None
    output = [char_to_idx[prefix[0]]] # output会记录prefix加上输出
    for t in range(num_chars + len(prefix) - 1):
        X = torch.tensor([output[-1]], device=device).view(1, 1)
        if state is not None:
            if isinstance(state, tuple): # LSTM, state:(h, c)
                state = (state[0].to(device), state[1].to(device))
            else:
                state = state.to(device)

        (Y, state) = model(X, state) # 前向计算不需要传入模型参数
        if t < len(prefix) - 1:
            output.append(char_to_idx[prefix[t + 1]])
        else:
            output.append(int(Y.argmax(dim=1).item()))
    return ''.join([idx_to_char[i] for i in output])
```

